

String Functions — Manipulation, Pattern Matching, and Formatting

Chapter 15 | *PostgreSQL Complete Guide* Module: 02_SQL_Basics | Difficulty: Beginner–Intermediate |

Estimated Reading Time: 45 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: Strings in PostgreSQL](#)
 - [Case Functions — UPPER, LOWER](#)
 - [Trimming — TRIM, LTRIM, RTRIM](#)
 - [Length Functions — LENGTH, CHAR_LENGTH](#)
 - [Substring and Position — SUBSTRING, POSITION](#)
 - [Replacement — REPLACE, REGEXP_REPLACE](#)
 - [Splitting — SPLIT_PART](#)
 - [Concatenation — CONCAT, ||, FORMAT](#)
 - [Padding — LPAD, RPAD](#)
 - [Regex Extraction — REGEXP_MATCHES, REGEXP_MATCH](#)
 - [ASCII Visual Diagrams](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Apply case, trim, length, and substring functions to transform text data
 - Build complex strings using CONCAT, ||, and FORMAT
 - Extract and manipulate substrings using POSITION, SUBSTRING, and SPLIT_PART
 - Perform regex-based replacements and extractions with REGEXP_REPLACE and REGEXP_MATCH
 - Choose the right string function for each use case and understand index implications
-

Theory: Strings in PostgreSQL

PostgreSQL stores text in three types:

- `TEXT` — variable-length, unlimited
- `VARCHAR(n)` — variable-length, max n characters
- `CHAR(n)` — fixed-length, padded with spaces

All three support the same string functions. PostgreSQL internally uses the encoding set at database creation (usually UTF-8) for multi-byte character support.

String Immutability

Strings in PostgreSQL are immutable. String functions return new strings; they do not modify in place. Every transformation creates a new value.

NULL Propagation

All string functions return NULL if any argument is NULL — **except** `CONCAT` (which ignores NULLs) and `COALESCE` (which handles NULLs directly).

Case Functions — UPPER, LOWER

```
UPPER(string)    -- convert to uppercase
LOWER(string)    -- convert to lowercase
INITCAP(string)  -- capitalize first letter of each word
```

```
SELECT UPPER('hello world');    -- 'HELLO WORLD'
SELECT LOWER('HELLO WORLD');    -- 'hello world'
SELECT INITCAP('alice smith');  -- 'Alice Smith'
SELECT INITCAP('ALICE SMITH');  -- 'Alice Smith'
```

Unicode Awareness

UPPER/LOWER are locale-aware and handle Unicode characters correctly when the database is UTF-8:

```
SELECT UPPER('straße');    -- 'STRASSE' (German ß -> SS)
```

Trimming — TRIM, LTRIM, RTRIM

```
TRIM([LEADING | TRAILING | BOTH] [chars FROM] string)
LTRIM(string [, chars])    -- remove from left
RTRIM(string [, chars])    -- remove from right
BTRIM(string [, chars])    -- remove from both ends
```

```
-- Default: remove spaces from both ends
SELECT TRIM(' hello ');    -- 'hello'
SELECT LTRIM(' hello ');   -- 'hello '
SELECT RTRIM(' hello ');   -- ' hello'

-- Remove specific characters
SELECT TRIM(LEADING '0' FROM '00123'); -- '123'
SELECT TRIM(TRAILING '/' FROM '/path/'); -- '/path'
SELECT TRIM(BOTH '*' FROM '***text***'); -- 'text'
```

```
-- LTRIM/RTRIM with character set (removes any character in the set)
SELECT LTRIM('xxxhello', 'x'); -- 'hello'
SELECT RTRIM('hello...', '.'); -- 'hello'
```

Length Functions — LENGTH, CHAR_LENGTH

```
LENGTH(string)      -- number of characters (for text/varchar)
CHAR_LENGTH(string)  -- SQL standard synonym for LENGTH
OCTET_LENGTH(string) -- number of bytes (important for multibyte chars)
BIT_LENGTH(string)   -- number of bits
```

```
SELECT LENGTH('hello');      -- 5
SELECT CHAR_LENGTH('hello'); -- 5
SELECT LENGTH('héello');     -- 5 (characters)
SELECT OCTET_LENGTH('héello'); -- 6 (bytes, because é is 2 bytes in UTF-8)

-- Length of NULL is NULL
SELECT LENGTH(NULL);         -- NULL

-- Use COALESCE for safe length check
SELECT LENGTH(COALESCE(description, '')) FROM products;
```

Substring and Position — SUBSTRING, POSITION

SUBSTRING

```
SUBSTRING(string FROM start FOR length) -- SQL standard syntax
SUBSTRING(string, start, length)         -- alternative syntax
SUBSTR(string, start, length)            -- alias
```

- `start` is 1-indexed (first character is position 1)
- If `start` is beyond end of string, returns empty string
- If `length` exceeds remaining characters, returns to end of string

```
SELECT SUBSTRING('PostgreSQL' FROM 1 FOR 4); -- 'Post'
SELECT SUBSTRING('PostgreSQL', 5);           -- 'greSQL' (no length = rest of
string)
SELECT SUBSTRING('PostgreSQL', 5, 3);        -- 'gre'
SELECT SUBSTR('PostgreSQL', 1, 4);           -- 'Post'

-- Regex-based SUBSTRING (PostgreSQL extension)
SELECT SUBSTRING('Phone: 555-1234' FROM '[0-9\-\-]+'); -- '555-1234'
```

POSITION

```
POSITION(substring IN string) -- SQL standard
STRPOS(string, substring)      -- PostgreSQL alias
```

Returns the starting position (1-indexed) of the first occurrence, or 0 if not found.

```
SELECT POSITION('SQL' IN 'PostgreSQL'); -- 8
SELECT STRPOS('PostgreSQL', 'SQL');    -- 8
SELECT POSITION('xyz' IN 'PostgreSQL'); -- 0 (not found)

-- Find position to extract dynamic substring
SELECT SUBSTRING(email FROM 1 FOR POSITION('@' IN email) - 1) AS local_part
FROM   users;
```

Replacement — REPLACE, REGEXP_REPLACE

REPLACE

```
REPLACE(string, from_text, to_text)
```

Replaces ALL occurrences of `from_text` with `to_text`. Case-sensitive.

```
SELECT REPLACE('Hello World', 'World', 'PostgreSQL'); -- 'Hello PostgreSQL'
SELECT REPLACE('aababab', 'ab', 'X');                 -- 'aXXX'
SELECT REPLACE('Hello', 'x', 'y');                    -- 'Hello' (no change, no
error)

-- Remove characters (replace with empty string)
SELECT REPLACE(phone, '-', '') FROM contacts;          -- remove dashes from phone
SELECT REPLACE(REPLACE(amount_text, '$', ''), ',', '') AS clean_amount FROM
invoices;
```

REGEXP_REPLACE

```
REGEXP_REPLACE(string, pattern, replacement [, flags])
```

Flags: `'g'` = replace all occurrences; `'i'` = case-insensitive.

```
-- Replace one or more spaces with a single space
SELECT REGEXP_REPLACE('Hello  World', '\s+', ' ', 'g'); -- 'Hello World'

-- Remove all non-digit characters from phone
SELECT REGEXP_REPLACE(phone, '[^0-9]', '', 'g') FROM contacts;

-- Replace digits with #
SELECT REGEXP_REPLACE('Card: 1234-5678', '[0-9]', '#', 'g'); -- 'Card: ####-####'
```

```
-- Case-insensitive replace
SELECT REGEXP_REPLACE('Hello hello HELLO', 'hello', 'hi', 'gi'); -- 'hi hi hi'
```

Splitting — SPLIT_PART

```
SPLIT_PART(string, delimiter, field_number)
```

Splits the string by `delimiter` and returns the `field_number` part (1-indexed). Returns empty string if field does not exist.

```
SELECT SPLIT_PART('one,two,three', ',', 1); -- 'one'
SELECT SPLIT_PART('one,two,three', ',', 2); -- 'two'
SELECT SPLIT_PART('one,two,three', ',', 3); -- 'three'
SELECT SPLIT_PART('one,two,three', ',', 4); -- '' (empty, no error)

-- Extract domain from email
SELECT SPLIT_PART(email, '@', 2) AS domain FROM users;

-- Extract year from date string
SELECT SPLIT_PART('2024-06-15', '-', 1) AS year; -- '2024'
```

STRING_TO_ARRAY and UNNEST

```
-- Split into array
SELECT STRING_TO_ARRAY('one,two,three', ','); -- ARRAY['one','two','three']

-- Split and expand to rows
SELECT UNNEST(STRING_TO_ARRAY('one,two,three', ',')) AS element;
```

Concatenation — CONCAT, ||, FORMAT

|| Operator

The `||` operator concatenates strings. **Returns NULL if any operand is NULL.**

```
SELECT 'Hello' || ' ' || 'World'; -- 'Hello World'
SELECT first_name || ' ' || last_name AS name FROM employees;
-- If last_name IS NULL: returns NULL (NULL propagation)
```

CONCAT Function

`CONCAT` ignores NULLs — it treats them as empty strings:

```
SELECT CONCAT('Hello', ' ', 'World'); -- 'Hello World'
SELECT CONCAT('Hello', NULL, 'World'); -- 'HelloWorld' (NULL ignored)
```

```
SELECT CONCAT(first_name, ' ', last_name) FROM employees; -- safe with NULLs
```

CONCAT_WS (With Separator)

```
SELECT CONCAT_WS(',', ' ', 'Alice', 'Smith', 'NYC'); -- 'Alice, Smith, NYC'
-- CONCAT_WS also ignores NULL values:
SELECT CONCAT_WS(',', ' ', 'Alice', NULL, 'NYC');      -- 'Alice, NYC'
```

FORMAT

FORMAT is like `sprintf` in C — uses positional placeholders:

```
FORMAT(format_string [, format_arg ...])
```

```
SELECT FORMAT('Hello, %s! You are %s years old.', 'Alice', 30);
-- 'Hello, Alice! You are 30 years old.'

-- %I = identifier (quoted if needed)
-- %L = literal (SQL-quoted)
-- %s = string (no quoting)
SELECT FORMAT('SELECT * FROM %I WHERE %I = %L', 'users', 'name', 'Alice');
-- 'SELECT * FROM users WHERE name = 'Alice''

-- Useful in dynamic SQL generation
DO $$
DECLARE
    tbl_name TEXT := 'employees';
    sql TEXT;
BEGIN
    sql := FORMAT('SELECT COUNT(*) FROM %I', tbl_name);
    EXECUTE sql;
END;
$$;
```

Padding — LPAD, RPAD

```
LPAD(string, length [, fill]) -- pad on the left to reach target length
RPAD(string, length [, fill]) -- pad on the right to reach target length
```

Default fill character is space. If string is longer than length, it is truncated.

```
SELECT LPAD('42', 6, '0');      -- '000042' (zero-padded number)
SELECT RPAD('hello', 10, '.');  -- 'hello.....'
SELECT LPAD('truncated', 5);    -- 'trunc' (truncated to 5 chars)
SELECT LPAD(order_id::TEXT, 8, '0') AS formatted_id FROM orders;
```

```
-- Right-align column of numbers
SELECT LPAD(salary::TEXT, 10) AS salary_col FROM employees;
```

Regex Extraction — REGEXP_MATCHES, REGEXP_MATCH

REGEXP_MATCH (PostgreSQL 10+)

Returns the first match as an array of captured groups (or NULL if no match):

```
REGEXP_MATCH(string, pattern [, flags])
```

```
SELECT REGEXP_MATCH('2024-06-15', '(\d{4})-(\d{2})-(\d{2})');
-- Returns: {2024,06,15}
```

```
SELECT (REGEXP_MATCH('Price: $49.99', '\$([0-9.]+)'))[1];
-- Returns: '49.99'
```

```
SELECT REGEXP_MATCH('no digits here', '\d+');
-- Returns: NULL
```

REGEXP_MATCHES (set-returning)

Returns all matches as a set of rows (one row per match):

```
REGEXP_MATCHES(string, pattern [, flags])
```

```
-- Find all hashtags in a tweet
SELECT REGEXP_MATCHES('Hello #world and #sql fans', '#(\w+)', 'g');
-- Row 1: {world}
-- Row 2: {sql}

-- Collect all matches into an array
SELECT ARRAY(SELECT REGEXP_MATCHES('a1b2c3', '[0-9]+', 'g'));
-- {{1},{2},{3}}
```

ASCII Visual Diagrams

SUBSTRING Indexing

```
String: 'PostgreSQL'
Index:  1234567890 (1-based)

SUBSTRING('PostgreSQL', 1, 4) --> 'Post'
      |<---->|
      1      4

SUBSTRING('PostgreSQL', 5, 3) --> 'gre'
```

```

      |<->|
      5  7

SUBSTRING('PostgreSQL', 5)    --> 'greSQL'
      |<----->|
      5          10

```

POSITION Search

```

String: 'PostgreSQL'
Position of 'SQL':

P o s t g r e S Q L
1 2 3 4 5 6 7 8 9 10
      ^
    POSITION = 8

```

LPAD/RPAD

```

LPAD('42', 6, '0'):
  Target length: 6
  Input: '42' (length 2)
  Need 4 more chars on left
  Result: '000042'

RPAD('hi', 8, '-*'):
  Target length: 8
  Input: 'hi' (length 2)
  Fill pattern '-*' repeated: -* -* -* ...
  Take first 6 chars: '-*-*-*'
  Result: 'hi-*-*-*'

```

SQL Examples

```

-- Example 1: UPPER / LOWER / INITCAP
SELECT UPPER(last_name), LOWER(email), INITCAP(city) FROM users;

-- Example 2: TRIM whitespace
SELECT TRIM(BOTH FROM '  hello  '); -- 'hello'
SELECT LTRIM('>>>hello', '>');      -- 'hello'

-- Example 3: LENGTH vs OCTET_LENGTH
SELECT LENGTH('café'),           -- 4 characters
       OCTET_LENGTH('café'); -- 5 bytes (UTF-8)

-- Example 4: SUBSTRING basic
SELECT SUBSTRING('Hello World', 7); -- 'World'
SELECT SUBSTRING('Hello World', 1, 5); -- 'Hello'

```



```

-- Example 5: SUBSTRING with regex
SELECT SUBSTRING('Order-2024-0042' FROM '[0-9]{4}'); -- '2024'

-- Example 6: POSITION / STRPOS
SELECT POSITION('.') IN 'user@example.com'; -- 12
SELECT STRPOS('user@example.com', '@'); -- 5

-- Example 7: Extract username from email
SELECT SUBSTRING(email FROM 1 FOR POSITION('@' IN email) - 1) AS username
FROM users;

-- Example 8: REPLACE all occurrences
SELECT REPLACE('banana', 'a', 'o'); -- 'bonono'

-- Example 9: REGEXP_REPLACE remove non-alphanumeric
SELECT REGEXP_REPLACE(phone_number, '[^0-9+]', '', 'g') FROM contacts;

-- Example 10: REGEXP_REPLACE normalize spaces
SELECT REGEXP_REPLACE(TRIM(description), '\s+', ' ', 'g') AS clean_desc
FROM products;

-- Example 11: SPLIT_PART for CSV column
SELECT SPLIT_PART(tags, ',', 1) AS first_tag FROM articles;

-- Example 12: STRING_TO_ARRAY + UNNEST
SELECT id, UNNEST(STRING_TO_ARRAY(tags, ',')) AS tag FROM articles;

-- Example 13: || operator (NULL-unsafe concatenation)
SELECT first_name || ' ' || COALESCE(middle_name || ' ', '') || last_name
FROM employees;

-- Example 14: CONCAT (NULL-safe)
SELECT CONCAT(first_name, ' ', last_name) FROM employees;

-- Example 15: CONCAT_WS with separator
SELECT CONCAT_WS(' | ', department, job_title, employee_id::TEXT) AS label
FROM employees;

-- Example 16: FORMAT for readable output
SELECT FORMAT('Employee %s earns $%s per year', full_name, annual_salary)
FROM compensation_view;

-- Example 17: LPAD for fixed-width output
SELECT LPAD(employee_id::TEXT, 6, '0') AS emp_code FROM employees;

-- Example 18: RPAD for column alignment
SELECT RPAD(product_name, 30, '.') || LPAD(unit_price::TEXT, 10) AS price_line
FROM products;

-- Example 19: REGEXP_MATCH extract first capture group
SELECT (REGEXP_MATCH(description, 'SKU:([A-Z0-9]+')))[1] AS sku
FROM products;

```

```
-- Example 20: REGEXP_MATCHES all matches as rows
SELECT product_id,
       REGEXP_MATCHES(tags, '[a-z]+', 'g') AS individual_tag
FROM   products;
```

Common Mistakes

1. **|| propagates NULL.** 'Hello' || NULL || 'World' returns NULL. Use `CONCAT()` or wrap nullable parts in `COALESCE`.
2. **SUBSTRING is 1-indexed, not 0-indexed.** `SUBSTRING('hello', 0, 3)` returns 'he' (not 'hel') because 0 is treated as 1, but the length is still counted from 0. Use `SUBSTRING(s, 1, 3)` for the first 3 characters.
3. **POSITION returns 0, not NULL, when not found.** Checking `WHERE POSITION('@' IN email) = NULL` always fails. Check `WHERE POSITION('@' IN email) = 0` or `WHERE email NOT LIKE '%@%'`.
4. **REPLACE is case-sensitive.** `REPLACE('Hello World', 'hello', 'hi')` returns the original string unchanged. Use `REGEXP_REPLACE(..., 'hello', 'hi', 'gi')` for case-insensitive replacement.
5. **SPLIT_PART returns empty string (not NULL) for missing fields.** `SPLIT_PART('a,b', ',', 5)` returns `' '`, not NULL. Test with `' ' != ''` not `IS NULL`.
6. **TRIM removes any character in the set, not a literal string.** `TRIM(BOTH 'abc' FROM 'abcXabc')` removes any occurrence of 'a', 'b', or 'c' from both ends, not the literal string "abc". To remove a literal prefix/suffix, use `REGEXP_REPLACE` or `SUBSTRING`.
7. **LENGTH counts characters, not bytes.** For byte-level operations on binary data, use `OCTET_LENGTH` and `bytea` type.

Best Practices

1. Use `CONCAT` or `CONCAT_WS` instead of `||` when any column could be NULL — they silently skip NULLs rather than returning NULL for the whole expression.
2. **Normalize input data at write time rather than at query time** — applying `LOWER/TRIM/REGEXP_REPLACE` in every `SELECT` is costly; store clean, normalized data and create functional indexes if needed.
3. **Create functional indexes for case-insensitive searches.** If you frequently search `WHERE LOWER(email) = LOWER(input)`, create `CREATE INDEX ON users (LOWER(email))` and query `WHERE LOWER(email) = 'search_term'`.
4. **Prefer `REGEXP_MATCH` over `REGEXP_MATCHES` when you only need the first match** — set-returning functions have overhead and require more complex handling when used inline.
5. **Use `FORMAT` with `%I` and `%L` for dynamic SQL** — never use string concatenation to build SQL (SQL injection risk). `FORMAT` properly quotes identifiers and literals.

6. **TRIM all user-provided text input before storing** — leading/trailing whitespace is the most common source of comparison failures and duplicate records.

7. **Benchmark string operations on large result sets** — functions like `REGEXP_REPLACE` with complex patterns can be CPU-intensive. Move transformation logic to application layer where possible.

Performance Considerations

Functional Indexes

```
-- Enable index use for case-insensitive search
CREATE INDEX idx_users_email_lower ON users (LOWER(email));

-- Query MUST match the index expression exactly
SELECT * FROM users WHERE LOWER(email) = LOWER('Search@example.com');
```

LIKE and Trigram Indexes

```
-- For LIKE '%pattern%' or ILIKE:
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_products_name_trgm ON products USING gin(product_name
gin_trgm_ops);

-- Now these are index-accelerated:
SELECT * FROM products WHERE product_name ILIKE '%wireless%';
SELECT * FROM products WHERE product_name ~ 'wire';
```

Avoid String Functions in WHERE on Large Tables

```
-- SLOW: function prevents index use
SELECT * FROM orders WHERE EXTRACT(YEAR FROM order_date::DATE) = 2023;

-- FAST: range comparison can use index
SELECT * FROM orders WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01';
```

Text Column Storage

For TEXT columns storing large values, PostgreSQL uses TOAST (The Oversized-Attribute Storage Technique). Very long strings are compressed and stored out-of-line. String functions that operate on TOASTed values incur decompression overhead.

Interview Questions

1. What is the difference between `||` and `CONCAT` for string concatenation?
2. How does `SUBSTRING(string, start, length)` handle 1-based indexing vs 0-based?
3. What does `POSITION` return when the substring is not found?
4. How would you remove all non-numeric characters from a phone number column?

5. What is the difference between `LENGTH` and `OCTET_LENGTH` ?
 6. How do you perform a case-insensitive string replacement?
 7. What does `SPLIT_PART('a,b,c', ',', 5)` return?
 8. How would you efficiently search for a substring anywhere in a `TEXT` column?
 9. Explain the difference between `REGEXP_MATCH` and `REGEXP_MATCHES`.
 10. Why should `FORMAT` with `%I` and `%L` be used instead of string concatenation for dynamic SQL?
-

Interview Answers

Q1: `||` vs `CONCAT`? `||` returns NULL if any operand is NULL (NULL propagation). `CONCAT` treats NULL arguments as empty strings and continues. Use `CONCAT` for nullable columns, `||` for guaranteed non-null values where strictness is desired.

Q2: `SUBSTRING 1-based indexing`? Position 1 is the first character. `SUBSTRING('hello', 1, 3)` returns 'hel'. If `start=0`, PostgreSQL treats it as 1 for the purpose of counting where to start, but the length is still from 0, so you get one fewer character than expected. Best practice: always start from 1.

Q3: `POSITION` when not found? Returns 0 (integer zero). It does not return NULL. This is a common source of bugs: `WHERE POSITION(x IN y) > 0` is the correct check for "found".

Q4: Remove non-numeric from phone? `REGEXP_REPLACE(phone, '[^0-9]', '', 'g')` — matches any character that is not a digit and replaces it with an empty string, globally.

Q5: `LENGTH` vs `OCTET_LENGTH`? `LENGTH` returns the number of characters. `OCTET_LENGTH` returns the number of bytes. They differ for multibyte characters (UTF-8): 'café' has `LENGTH` = 4 but `OCTET_LENGTH` = 5 because 'é' is 2 bytes.

Q6: Case-insensitive replacement? `REGEXP_REPLACE(string, 'pattern', 'replacement', 'gi')` — the 'i' flag makes it case-insensitive, 'g' replaces all occurrences. Standard `REPLACE` is always case-sensitive.

Q7: `SPLIT_PART` beyond bounds? Returns an empty string `''`, not NULL and not an error.

Q8: Efficient substring search? Enable `pg_trgm` extension and create a GIN index with `gin_trgm_ops`. This enables index use for `LIKE '%term%'`, `ILIKE '%term%'`, and `~` regex operators.

Q9: `REGEXP_MATCH` vs `REGEXP_MATCHES`? `REGEXP_MATCH` returns the first match as a single array (or NULL if no match) — one result per row. `REGEXP_MATCHES` is a set-returning function that returns one row per match (useful with the 'g' global flag). Use `REGEXP_MATCH` for the first match, `REGEXP_MATCHES` to iterate all matches.

Q10: `FORMAT` with `%I` and `%L`? String concatenation to build SQL creates SQL injection vulnerabilities: `'SELECT * FROM ' || user_input` could be exploited. `%I` (identifier) and `%L` (literal) in `FORMAT` automatically apply the correct quoting (double-quotes for identifiers, single-quotes with escaping for literals), making the SQL safe.

Hands-on Exercises

Setup:

```
CREATE TABLE contacts (  
    contact_id SERIAL PRIMARY KEY,  
    full_name TEXT,
```

```

    email        TEXT,
    phone        TEXT,
    address      TEXT
);

INSERT INTO contacts (full_name, email, phone, address) VALUES
(' Alice Smith ', 'ALICE@EXAMPLE.COM', '(555) 123-4567', '123 Main St,
Springfield, IL 62701'),
('bob jones',      'bob@test.org',      '555.987.6543',   '456 Oak Ave, Chicago,
IL 60601'),
('CAROL WILLIAMS', 'carol@example.net', '+1-800-555-0100', '789 Pine Rd, Rockford,
IL 61101'),
('David Brown',    NULL,                '5559991234',     NULL),
('Eve Taylor',     'eve@sample.io',    '(555)246-8101',  '321 Elm St, Peoria, IL
61602');

```

Exercise 1: Clean the full_name column: trim whitespace and apply INITCAP.

Exercise 2: Extract the username (part before @) and domain (part after @) from the email column. Handle NULL emails with 'N/A'.

Exercise 3: Normalize the phone column to digits only (remove all non-numeric characters). Format the result as (NNN) NNN-NNNN using LPAD/SUBSTRING.

Exercise 4: Extract the city and state from the address column (format: street, city, state zip). Handle NULL addresses.

Exercise 5: Build a formatted label string: ID: XXXXXX | Name: Full Name | Email: email using FORMAT and CONCAT_WS.

Solutions

```

-- Exercise 1
SELECT contact_id,
       INITCAP(TRIM(full_name)) AS cleaned_name
FROM   contacts;

-- Exercise 2
SELECT contact_id,
       COALESCE(SPLIT_PART(email, '@', 1), 'N/A') AS username,
       COALESCE(SPLIT_PART(email, '@', 2), 'N/A') AS domain
FROM   contacts;

-- Exercise 3
SELECT contact_id,
       phone AS original_phone,
       REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS digits_only,
       '(' || SUBSTRING(REGEXP_REPLACE(phone, '[^0-9]', '', 'g'), 1, 3) || ') ' ||
       SUBSTRING(REGEXP_REPLACE(phone, '[^0-9]', '', 'g'), 4, 3) || '-' ||
       SUBSTRING(REGEXP_REPLACE(phone, '[^0-9]', '', 'g'), 7, 4) AS
formatted_phone

```

```

FROM contacts;

-- Exercise 4
SELECT contact_id,
       COALESCE(SPLIT_PART(address, ' ', 2), 'Unknown') AS city,
       COALESCE(SPLIT_PART(SPLIT_PART(address, ' ', 3), ' ', 1), 'Unknown') AS
state
FROM contacts;

-- Exercise 5
SELECT FORMAT('ID: %s | Name: %s | Email: %s',
            LPAD(contact_id::TEXT, 6, '0'),
            INITCAP(TRIM(full_name)),
            COALESCE(email, 'N/A')) AS label
FROM contacts;

```

Advanced Notes

Text Search with `tsvector` and `tsquery`

For full-text search, PostgreSQL's built-in text search is more powerful than LIKE:

```

SELECT * FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ to_tsquery('postgresql &
index');

```

String Aggregation

```

-- Concatenate values from multiple rows
SELECT department_id,
       STRING_AGG(last_name, ' ' ORDER BY last_name) AS employee_list
FROM employees
GROUP BY department_id;

-- Distinct values only
SELECT STRING_AGG(DISTINCT category, ' ') FROM products;

```

Text Pattern Matching with `pg_trgm` Similarity

```

CREATE EXTENSION pg_trgm;

-- Similarity score between two strings (0.0 to 1.0)
SELECT SIMILARITY('hello', 'helo');    -- ~0.57

-- Find strings similar to a search term
SELECT product_name FROM products
WHERE SIMILARITY(product_name, 'widgt') > 0.3;

```

encode and decode for Binary Data

```
SELECT ENCODE('binary data'::bytea, 'base64'); -- Base64 encode
SELECT ENCODE('binary data'::bytea, 'hex');    -- Hex encode
SELECT DECODE('68656c6c66', 'hex');           -- Hex decode
```

Cross-References

- **Previous:** [05 null handling.md](#) — NULL in string concatenation
- **Next:** [07 numeric functions.md](#) — Numeric manipulation
- **Related:** [04 filtering and operators.md](#) — LIKE, ILIKE, regex operators
- **Related (03 Intermediate):** [03_Intermediate_SQL/07_full_text_search.md](#) — tsvector, tsquery
- **Related (03 Intermediate):** [03_Intermediate_SQL/06_indexes.md](#) — Functional indexes, trigram indexes